

ZAP 0.2.0

Smart Contract Security Assessment

December 2025

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Missing Validation in find_x0_and_delta_x of CurveHandler and BidAskHandler	6
4.2	Precision Loss May Lead to Inaccurate Results in CurveHandler.find_x0_and_delta_x	7
4.3	Possible Unexpected swap_amount in calculate_swap_amount	8
4.4	Rounding-Down Division May Allow Larger Price Diff	9
4.5	Informational and Undetermined Issues	10
5	Disclaimer	13



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *Meteora ZAP* smart contracts, starting on November 20th, 2025, and concluding on December 2nd, 2025.

Project Overview

The ZAP program delivers one-click liquidity zaps. Zap Out enables users to exit LP positions or claim fees and immediately route a configured portion through whitelisted AMMs (e.g., DAMM V2, DLMM, Jupiter). Zap In lets users pre-record intended dual/single-asset contributions in a ledger; the program then performs the necessary single-sided swaps and adds or rebalances liquidity in DAMM V2 or DLMM. The goal is to collapse multi-step swap + LP workflows into a minimal set of on-chain calls, reducing operational overhead and slippage risk.

Audit Scope

The assessment scope contains mainly the smart contracts of the zap program for the *Meteora ZAP* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Meteora ZAP
 - Codebase: <https://github.com/MeteoraAg/zap-program>
 - Release 0.2.0: [PR-22](#)
 - Commit Hash: 3162ebb129e9cfd50fba4300758288c118ce4d33

The issues described in this report have been resolved by the following commit:

- Meteora ZAP
 - Codebase: <https://github.com/MeteoraAg/zap-program>
 - Commit Hash: 799416cabb187045f0cbbb732b094958049c033d

We listed the files we have audited below:

- Meteora ZAP Release 0.2.0
 - programs/zap/src/math/dlmm_price_math.rs
 - programs/zap/src/math/price_math.rs
 - programs/zap/src/math/safe_math.rs
 - programs/zap/src/constants.rs
 - programs/zap/src/instructions/ledger_instructions.rs
 - programs/zap/src/instructions/ix_zap_in_damm_v2.rs
 - programs/zap/src/instructions/zap_in_dlmm/ix_zap_in_dlmm_for_uninitialized_position.rs
 - programs/zap/src/instructions/zap_in_dlmm/ix_zap_in_dlmm_for_initialized_position.rs



- programs/zap/src/utils/token.rs
- programs/zap/src/utils/damm_v2_utils.rs
- programs/zap/src/utils/dlmm_utils.rs
- programs/zap/src/error.rs
- programs/zap/src/state/user_ledger.rs
- programs/zap/src/lib.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 0 medium issue
- 4 low issues
- 6 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Missing Validation in find_x0_and_delta_x of CurveHandler and BidAskHandler	Low	Fixed
02	Precision Loss May Lead to Inaccurate Results in CurveHandler.find_x0_and_delta_x	Low	Fixed
03	Possible Unexpected swap_amount in calculate_swap_amount	Low	Fixed
04	Rounding-Down Division May Allow Larger Price Diff	Low	Fixed
05	Possible Integer Overflow in calculate_swap_amount	Informational	Fixed
06	Missing Parameter Validation in zap_in_dlmm_for_*_position IX	Informational	Fixed
07	Possible Missing Validation on ZapInDlmmForUninitializedPositionCtx.position	Informational	Fixed
08	Typo	Informational	Fixed
09	Unsupported BaseFeeMode in damm_v2_utils	Informational	Fixed
10	Missing Type Validation for UpdateLedgerBalanceAfterSwapCtx.token_account	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Missing Validation in `find_x0_and_delta_x` of `CurveHandler` and `BidAskHandler`

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

In `CurveHandler.find_x0_and_delta_x`, there is no check for `min_delta_id == max_delta_id`. If these two values in the input parameters are equal, this will directly cause a division-by-zero exception when computing `x0` in subsequent calculations because `b == c`:

```
325     let x0 = U256::from(amount_x)
326         .safe_shl(64)?
327         .safe_div(b.safe_sub(c))?);
```

[programs/zap/src/ultils/dlmm_utils.rs#L325-L327](#)

The same issue also exists in `BidAskHandler.find_x0_and_delta_x`.

In particular, when `min_delta_id == max_delta_id == 0`, a division-by-zero exception will also occur when computing `c_delta`:

```
320     let c_delta = U256::from(m).safe_mul(pm)?.safe_div(U256::from(m2))?;
```

[programs/zap/src/ultils/dlmm_utils.rs#L320-L320](#)

Impact

This division-by-zero exception will cause the `Curve` and `BidAsk` strategies to never be able to support the case where `min_delta_id == max_delta_id`.

Recommendation

It is recommended to add separate handling for the case where `min_delta_id == max_delta_id`.

Mitigation Review Log

Fixed in the commit `680b172c8f35343e7dbf73583922cbfe0f4679ea`.



4.2 Precision Loss May Lead to Inaccurate Results in CurveHandler.find_x0_and_delta_x

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Precision

Description

In `CurveHandler.find_x0_and_delta_x`, the value of `c` is repeatedly computed and updated in a loop:

```
314     for m in m1..m2 {
315         let bin_id = active_id.safe_add(m)?;
316         let pm = U256::from(get_price_from_id(bin_id.neg(), bin_step)?);
317
318         b = b.safe_add(pm)?;
319
320         let c_delta =
321             ↪ U256::from(m).safe_mul(pm)?.safe_div(U256::from(m2))?;
322
323         c = c.safe_add(c_delta)?;
324     }
```

[programs/zap/src/ultils/dlmm_utils.rs#L314-L323](#)

However, in this loop, each time `c_delta` is computed, a bit of precision may be lost due to integer division rounding down. Over the entire loop, precision will be lost a total of `(max_delta_id - min_delta_id)` times.

Impact

As a result, the computed `c` will be smaller than it should be, which in turn causes the final `x0` and `delta_x` to deviate slightly from the expected values.

Recommendation

It is recommended to defer the division when computing `C` until after the loop and perform it only once at the end, as indicated by the formula in the code comment:

```
C = (m1 * p(m1) + ... + m2 * p(m2)) / m2
```

Mitigation Review Log

Fixed in the commit `db5e207a0a741c0270a6b6347a3c87355cc42eb7`.



4.3 Possible Unexpected swap_amount in calculate_swap_amount

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

When `calculate_swap_amount` computes the precise `swap_amount` via binary search, it may return a highly inaccurate `swap_amount`.

When retrieving the return values of `calculate_swap_result` and `validate_swap_result` inside the loop, if the return value is `Error`, the loop will break immediately and directly return `swap_amount`.

```
439         } else {
440             #[cfg(test)]
441             println!("can't validate swap result {}", _i);
442
443             break; // if we can't validate swap result, then just break
444         }
445     } else {
446         #[cfg(test)]
447         println!("can't simulate swap result {}", _i);
448
449         break; // if we can't simulate swap result, then just break
450     }
```

[programs/zap/src/ultils/damm_v2_utils.rs#L439-L450](#)

At this point, if the loop body has already executed at least once, then `swap_amount` is greater than 0, and the `zap_in_damm_v2` IX will directly use this function's return value as the `swap_amount` for the subsequent swap, but this value is unexpected.

Impact

An inaccurate `swap_amount` may cause the program to fail to execute the most precise swap for the user, leaving the user with more Token A/B remaining than expected, and `zap_in_damm_v2` therefore does not achieve the intended effect.

Recommendation

It is recommended that when an error occurs inside the loop, the error should be propagated and reported to the outermost caller, rather than directly using an intermediate search result.

Mitigation Review Log

Fixed in the commit `db5e207a0a741c0270a6b6347a3c87355cc42eb7`.



4.4 Rounding-Down Division May Allow Larger Price Diff

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Precision

Description

In the `zap_in_damm_v2` IX, `price_diff_bps` is computed using floor division.

```
490 let price_diff_prod =  
    ↪ U192::from(price_diff).safe_mul(U192::from(MAX_BASIS_POINT))?;  
491  
492 let price_diff_bps =  
    ↪ price_diff_prod.safe_div(U192::from(pre_sqrt_price))?;
```

[programs/zap/src/utils/damm_v2_utils.rs#L490-L492](#)

Then `price_diff_bps` is compared with `max_sqrt_price_change_bps`. Due to this flooring, a price diff that is actually larger than `max_sqrt_price_change_bps` may still pass the subsequent check.

```
215 require!(  
216     sqrt_price_change_bps <= max_sqrt_price_change_bps,  
217     ZapError::ExceededSlippage  
218 );
```

[programs/zap/src/instructions/ix_zap_in_damm_v2.rs#L215-L218](#)

Impact

This rounding down division and check may result in a larger-than-expected `price_diff`.



Proof of Concept

```
pre_sqrt_price = 100000
max_sqrt_price_change_bps = 1
expected_price_diff = 100000 * 1 / 10000 = 10

// case 1:
price_diff_1 == 10
price_diff_bps_1 = 10 * 10000 / 100000 = 1
// price_diff_bps_1 <= max_sqrt_price_change_bps
// price_diff_1 <= expected_price_diff

// case 2:
price_diff_2 == 19
price_diff_bps_2 = 19 * 10000 / 100000 = 1
// price_diff_bps_2 <= max_sqrt_price_change_bps
// [X] price_diff_2 > expected_price_diff
```

Recommendation

It is recommended to change the division to use ceiling instead, or to change the check to:

```
price_diff * 10000 <= pre_sqrt_price * max_sqrt_price_change_bps
```

Mitigation Review Log

Fixed in the commit db5e207a0a741c0270a6b6347a3c87355cc42eb7.

4.5 Informational and Undetermined Issues

Possible Integer Overflow in calculate_swap_amount

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

When `calculate_swap_amount` computes `swap_amount` via binary search, there may be an integer overflow.

```
391 let amount_in =
    max_swap_amount.safe_add(min_swap_amount)?.safe_div(2)?;
```

[programs/zap/src/ultils/damm_v2_ultils.rs#L391-L391](#)

It is recommended to use a safer implementation to avoid potential integer overflow:

```
min + (max - min) / 2 or min + ((max - min) >> 1)
```



Missing Parameter Validation in zap_in_dlmm_for_*_position IX

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Data Validation

In the `zap_in_dlmm_for_initialized_position` and `zap_in_dlmm_for_uninitialized_position` IX, there is no check to ensure that the input `min_delta_id ≤ max_delta_id`. If the two parameters provided by the user do not satisfy this requirement, a division-by-zero error or a type conversion failure may occur in subsequent execution. It is recommended to add a check on the relationship between these two values in the IX to ensure that no unexpected errors are triggered during later computations.

Possible Missing Validation on ZapInDlmmForUninitializedPositionCtx.position

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

In `ZapInDlmmForUninitializedPositionCtx`, it is explicitly required that `position ≠ owner` to prevent the user from mistakenly setting the owner as the position. However, in this IX, in addition to `position` and `owner` being signers, `rent_payer` is also a signer. Since the check `position ≠ owner` has already been added, it would also be reasonable to add `position ≠ rent_payer`.

Typo

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

There are several typos in the code comments and filenames.

- Filename
 - `damm_v2_utils.rs` ⇒ `damm_v2_utils.rs`
- Code comments
 - `advoid` ⇒ `avoid`: [programs/zap/src/utills/dlmm_utils.rs#L254-L254](#)
 - `becase` ⇒ `because`: [programs/zap/src/utills/dlmm_utils.rs#L254-L254](#)
 - `equa` ⇒ `equal`: [programs/zap/src/utills/dlmm_utils.rs#L254-L254](#)
 - `possitive` ⇒ `positive`: [programs/zap/src/utills/dlmm_utils.rs#L274-L274](#)
 - `x0` ⇒ `delta_x`: [programs/zap/src/utills/dlmm_utils.rs#L395-L395](#)
 - Missing unit for CU consumption: [programs/zap/src/utills/damm_v2_utils.rs#L388-L389](#)

Unsupported BaseFeeMode in damm_v2_utils

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA



In `damm_v2_utils's get_fee_handler` , if converting `base_fee_mode` into `BaseFeeMode` fails, then by default `base_fee_numerator` only uses `cliff_fee_numerator`. For a base fee mode that is currently unknown, if the zap does not support it yet, it is better to return an error here instead of directly using a potentially incorrect `cliff_fee_numerator`.

Missing Type Validation for `UpdateLedgerBalanceAfterSwapCtx.token_account`

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Data Validation

In the `update_ledger_balance_after_swap` IX, the passed-in `token_account` is an `UncheckedAccount` . In this case, even if the user passes in an arbitrary account whose type is not a Token Account, as long as there is data in `AccountInfo.data[64..72]` , this IX can still complete successfully. It is recommended to add an Anchor constraint on `UpdateLedgerBalanceAfterSwapCtx.token_account` to explicitly require it to be of type `TokenAccount` .



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

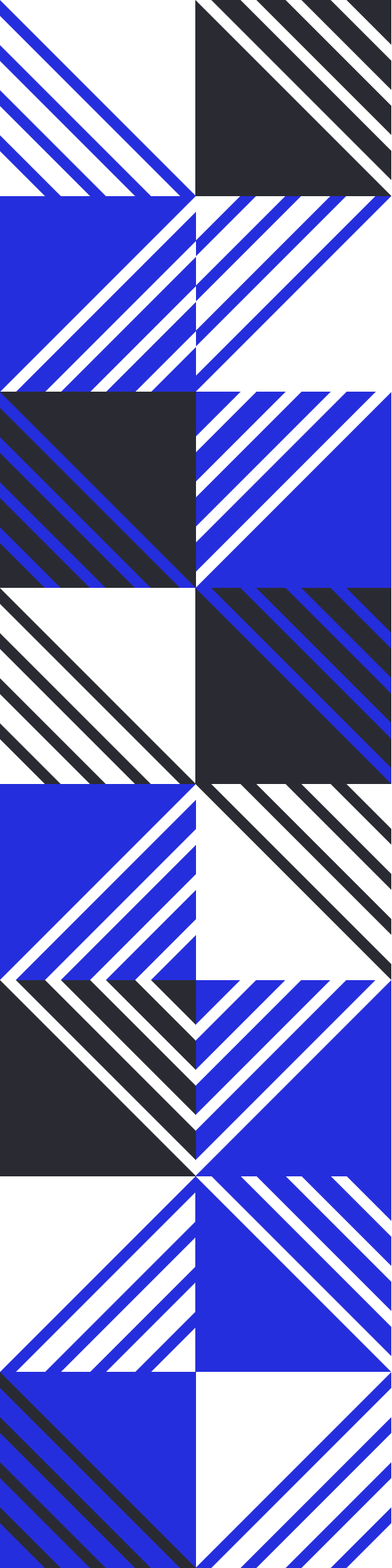
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs